

Instruction: Lists and Dictionaries

Learning Objective

In this lesson, you will learn about two fundamental data structures in Python: **Lists** and **Dictionaries**.

These structures form the foundation of how data is stored and analyzed in real-world applications.

What is a List?

A **list** is an ordered collection of items in Python. Lists can store multiple values, including numbers, strings, and even other lists.

Creating a List

To create a list, use square brackets `[ ]` and separate the items with commas.

```
# Example
numbers = [1, 2, 3, 4, 5]
fruits = ["apple", "banana"]
```

Example Code & Output

```
# Mixed list example
mixed_list = [10, "hello", 3.5, True]
print(mixed_list)

>>> Expected Output:
[10, 'hello', 3.5, True]
```

Accessing Elements in a List

Each item in a list has an **index** starting from 0. You can access individual items using their index.

Example Code

```
# Define a list
fruits = ["apple", "banana", "cherry"]

# Access items by index
print(fruits[0]) # First item
print(fruits[1]) # Second item
print(fruits[-1]) # Last item
```

Expected Output

```
apple
banana
cherry
```

Tip: Using -1 lets you access the last item in the list, -2 the second-last, and so on.

Modifying Lists

You can change, add, and remove items in a list using various Python methods and indexing.

Example Code

```
# Initial list
fruits = ["apple", "banana", "cherry"]

# Modifying elements
fruits[1] = "blueberry"
fruits.append("orange")
fruits.remove("apple")

# Output result
print(fruits)
```

Expected Output

```
['blueberry', 'cherry', 'orange']
```

Looping Through a List

You can use a **for loop** to go through each item in a list efficiently.

Example Code

```
# Define a list
fruits = ["apple", "banana", "cherry"]

# Loop through the list
for fruit in fruits:
    print(fruit)
```

Expected Output

```
apple
banana
cherry
```

What is a Dictionary?

A **dictionary** is a collection of key-value pairs. Unlike lists, they use **keys** to store and retrieve values instead of indices.

Creation: Use curly braces { } with keys and values separated by colons :.

Example Code

```
# Dictionary storing student info
student = {
    "name": "Alice",
    "age": 17,
    "grade": "A"
}

print(student)
```

Expected Output

```
{'name': 'Alice', 'age': 17, 'grade': 'A'}
```

Accessing Dictionary Values

To access a value, use its **key** inside square brackets `[ ]` or the `.get()` method.

Example Code

```
# Access using key
print(student["name"])

# Access using .get()
print(student.get("age"))
```

Expected Output

```
Alice
17
```

Modifying Dictionaries

You can **add**, **change**, and **remove** key-value pairs in a dictionary.

Example Code

```
# Adding a new key-value pair
student["school"] = "High School"

# Changing a value
student["grade"] = "A+"

# Removing a key-value pair
del student["age"]

print(student)
```

Expected Output

```
{'name': 'Alice', 'grade': 'A+',
'school': 'High School'}
```


Looping Through a Dictionary

You can loop through **keys**, **values**, or **both** in a dictionary using the `.items()` method.

Example Code

```
# Loop through key-value pairs
for key, value in student.items():
    print(f"{key}: {value}")
```

Expected Output

```
name: Alice
grade: A+
school: High School
```

Lists vs. Dictionaries: When to Use Each?

Feature	List	Dictionary
Order	Maintains order	No guaranteed order (Python 3.6+ maintains order)
Access	By index (e.g., <code>list[0]</code>)	By key (e.g., <code>dict["name"]</code>)
Modifications	Can add/remove items	Can add/remove key-value pairs
Best Use Case	Ordered collections of items	Data with labeled attributes

Reflection Questions

When would you use a **list** instead of a **dictionary**?

What are the advantages of using a dictionary over a list?

How can you modify a dictionary dynamically while running a program?

Activity: Lists & Dictionaries

Objective

In this activity, you will practice working with lists and dictionaries in Python. You'll create, modify, and access data using these structures to reinforce their importance in programming.

Tasks to Complete

- Initialize a list and a dictionary
- Access elements using indices and keys
- Modify values and update structures
- Iterate through collections with loops

Pro-Tips

Lists: Best for ordered collections where position matters.

Dictionaries: Use for labeled data mapping keys to values.

Part 1: Working with Lists

Task 1: Creating and Accessing Lists

Write a Python program that does the following:

- Create a list called **favorite_movies** with 5+ titles.
- Print the first and last movie using indexing.
- Print the entire list to confirm inclusion.

Example Code

```
# Define the list
favorite_movies = ["Inception", "The Dark Knight",
                  "Parasite", "Whiplash", "Interstellar"]

# Access by index
print(f"My first favorite movie: {favorite_movies[0]}")
print(f"My last favorite movie: {favorite_movies[-1]}")

# Print full list
print(f"My favorite movies: {favorite_movies}")
```

Expected Output

```
My first favorite movie: Inception
My last favorite movie: Interstellar
```

```
My favorite movies: ['Inception', 'The Dark
Knight', 'Parasite', 'Whiplash',
'Interstellar']
```

Part 1: Working with Lists

Task 2: Modifying Lists

Update your `favorite_movies` list by following these steps:

- Change the second movie to a new choice
- Add another movie to the end
- Remove the third movie and print the final list

Example Code

```
# Change the second movie
favorite_movies[1] = "The Matrix"

# Add a new movie at the end
favorite_movies.append("Titanic")

# Remove the third movie
del favorite_movies[2]

print(favorite_movies)
```

Expected Output

```
['Inception', 'The Matrix', 'Whiplash',  
'Interstellar', 'Titanic']
```

Part 2: Working with Dictionaries

Task 3: Creating and Accessing Dictionaries

Write a Python program that does the following:

- Create a dictionary called **student_info** with keys for 'name', 'age', and 'major'.
- Print the student's name using its key.
- Add a new key-value pair for 'graduation_year' and print the dictionary.

Example Code

```
# Initialize dictionary
student = {"name": "Alice", "age": 21, "major": "CS"}

# Access value by key
print(f"Student Name: {student['name']}")

# Add new key-value pair
student["grad_year"] = 2025
print(student)
```

Expected Output

Student Name: Alice

```
{'name': 'Alice', 'age': 21, 'major': 'CS',
'grad_year': 2025}
```

Task 4: Creating and Accessing a Dictionary

1. Create a dictionary `student_info` with keys: 'name', 'age', 'grade', 'favorite_subject'.
2. Print the student's name and favorite subject using dictionary keys.

Example Code

```
# Initialize dictionary
student_info = {
    "name": "Alice", "age": 16,
    "grade": "11th",
    "favorite_subject": "Math"
}

# Access values
print(f"Student Name: {student_info['name']}")
print(f"Favorite Subject:
{student_info['favorite_subject']}")
```

Expected Output

Student Name: Alice

Favorite Subject: Math

Part 2: Working with Dictionaries

Task 5: Modifying and Adding to a Dictionary

- Change the "grade" value to "12th".
- Add a new key-value pair: "hobby" → "Reading".
- Remove the "age" key from the dictionary.
- Print the updated dictionary.

Example Code

```
# Update grade
student_info["grade"] = "12th"

# Add a hobby
student_info["hobby"] = "Reading"

# Remove age
del student_info["age"]

print(student_info)
```

Expected Output

```
{'name': 'Alice', 'grade': '12th',
'favorite_subject': 'Math', 'hobby':
'Reading'}
```


Part 2: Working with Dictionaries

Task 6: Looping Through a Dictionary

Write a for loop to print each key and value in the `student_info` dictionary.

Example Code

```
# Loop through key-value pairs
for key, value in student_info.items():
    print(f"{key}: {value}")
```

Expected Output

```
name: Alice
grade: 12th
favorite_subject: Math
hobby: Reading
```

Part 2: Working with Dictionaries

Bonus Challenge: Combining Lists and Dictionaries

- Create a list called `class_roster` containing 3 dictionaries.
- Each dictionary should have "name", "age", and "favorite_subject".
- Loop through and print each student's name and favorite subject.

Example Code

```
class_roster = [  
    {"name": "Alice", "age": 16, "fav": "Math"},  
    {"name": "Bob", "age": 17, "fav": "Science"},  
    {"name": "Charlie", "age": 16, "fav": "History"}  
]  
  
for student in class_roster:  
    name = student['name']  
    subj = student['fav']  
    print(f"{name}'s favorite is {subj}.")
```

Expected Output

```
Alice's favorite subject is Math.  
Bob's favorite subject is Science.  
Charlie's favorite subject is History.
```

Part 3: Conclusion & Reflection

Reflection Questions

- How are lists and dictionaries different from each other?
- When would you use a list vs. a dictionary?
- Why is it useful to loop through a list or a dictionary?

Lists and dictionaries allow us to structure and organize data, making it possible to analyze, transform, and work with real-world datasets efficiently.

- **Save and submit your completed notebook**

Part 1: Foundational Reading

Activity: Read Chapter 4 - "Lists"

From "Automate the Boring Stuff with Python"

In this chapter, you will learn how to use and manipulate lists in Python. It's important to get these in-depth looks and practice from multiple authors. Getting multiple views and explanations on a topic helps you to create a more complete understanding of its function.

Access Reading: Chapter 4 Link →

Part 1: Foundational Reading

Activity: Read Chapter 5 - "Dictionaries and Structuring Data"

From "Automate the Boring Stuff with Python"

In this activity, read up on and learn more about the different data structures in Python. Understanding what you will be working with is essential to being able to utilize them for good data analysis.

Access Reading: Chapter 5 Link →

Python Basics

Computer Science

Activity - String Manipulation

In this activity, you will accomplish the following:

- Take in a sentence as an input
- Replace all the spaces with ...
- Print the new sentence as an output.

- **Save and submit your completed notebook**